

Scalability & Future Plan

Date	8 July 2026
Team ID	XXXXXX
Project Name	Smart Lender
Maximum Marks	1 Mark

Scalability & Future Plan:

This document outlines how the current project solution can be scaled to handle larger user bases, increased data loads, or extended features in the future. It also captures the team's roadmap for enhancing and evolving the project beyond its current state.

Current System Limitations:

S.No	Limitation	Impact	Priority to Address(High / Medium / Low)
1	Local In-Memory Processing	The Flask application currently runs on a single local server, meaning it cannot handle concurrent requests from multiple bank analysts or high volumes during peak periods.	High
2	File-Based Model Loading	The trained XGBoost model (.pkl or .json) is stored locally within the application directory, making updates and version control inefficient if multiple servers are deployed.	Medium
3	Static Training Data	The system relies on a historical, static dataset. It does not automatically learn from new loan applications or adapt to shifting economic trends/credit behaviors without manual retraining.	High

Scalability Plan:

S.No	Scalability Aspect	Current State	Proposed Upgrade / Solution
1	User Load	Single-threaded Flask application running on a local server instance, making it suitable only for individual testing or single-officer use.	Deploy the Flask application using Gunicorn/WSGI containers behind an NGINX load balancer on IBM Cloud to scale application instances dynamically based on web traffic.
2	Data Storage	Uses a static, historical dataset originally sourced from Google Sheets and processed locally as a raw CSV file.	Migrate data storage to an enterprise cloud database like IBM Cloudant or PostgreSQL to securely catalog new applicant entries, track decisions, and warehouse training data.
3	Performance	Synchronous model inference; the web server process directly executes the XGBoost pipeline, freezing the thread until computation finishes.	Introduce an asynchronous task queue framework like Celery with Redis to batch-evaluate high-volume applications during peak hours without impacting UI responsiveness.
4	Security	Open, unauthenticated local web form transmitting raw financial and personal applicant data	Restrict system access by enforcing SSL/HTTPS encryption, implementing Role-Based Access Control (RBAC) for loan officers, and introducing

S.No	Scalability Aspect	Current State	Proposed Upgrade / Solution
		over unencrypted HTTP channels.	JSON Web Tokens (JWT) for secure API endpoints.

Future Roadmap:

Phase	Planned Feature / Enhancement	Target Timeline	Expected Impact
Phase 2	Automated Data Pipelines & Bureau API Integration	Month 1 - 2	Connect the backend directly to live credit bureau APIs to pull real-time applicant credit scores, minimizing manual entry errors and streamlining evaluation data streams.
Phase 3	Explainable AI (XAI) Dashboard Integration	Month 3 - 4	Embed frameworks like SHAP or LIME into the Flask interface to show credit officers exactly which risk factors led to a denial, ensuring full regulatory transparency and compliance.
Phase 4	Advanced Alternative Data Risk Assessment	Month 5 - 6	Enhance the classification pipeline to evaluate alternative data streams (e.g., utility bills, cash flow patterns, and transaction histories) to safely extend credit eligibility to unbanked individuals.